

Documentação Técnica — Desafio 2

1. Objetivo

Implementar e demonstrar **replicação lógica** entre **2 servidores PostgreSQL** usando o modelo de **publicação e subscrição** (disponível desde o PostgreSQL 10). O projeto cobre os 4 requisitos: 2 servidores, publicações/subscrições para tabelas específicas, replicação seletiva e inserções/atualizações nos dois lados. Tudo é reproduzível via Docker.

2. Arquitetura

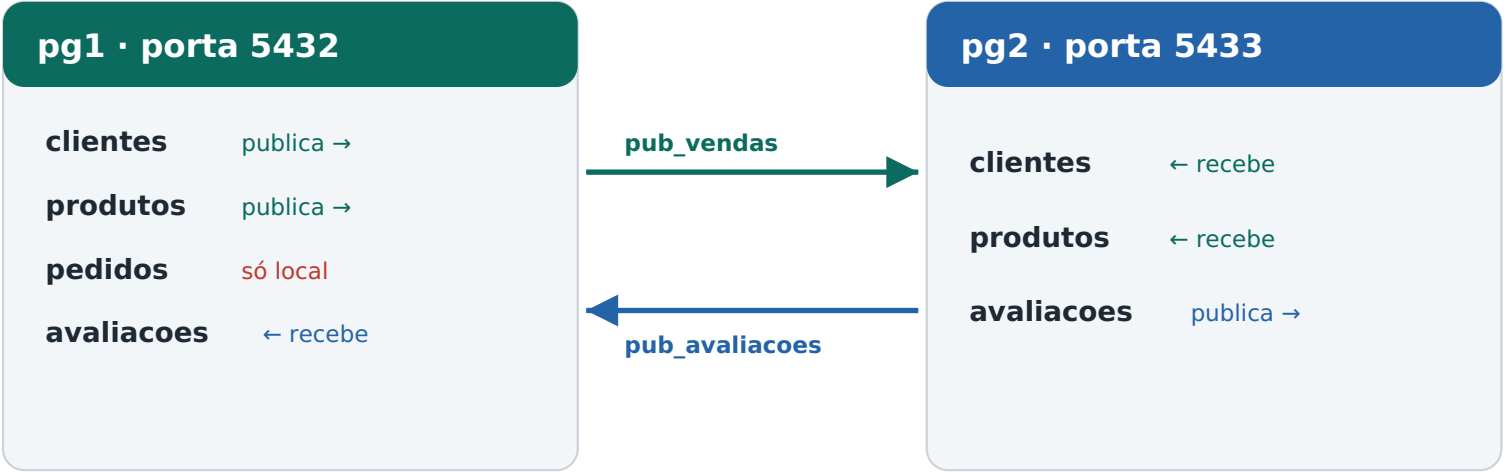


Figura 1 — pg1 publica clientes/produtos; pg2 publica avaliacoes; pedidos fica de fora (seletiva).

Os dois servidores são **independentes e graváveis**. pg1 publica clientes e produtos (que pg2 recebe); pg2 publica avaliacoes (que pg1 recebe). A tabela pedidos existe só no pg1 e não é publicada — é a prova da replicação seletiva.

3. Conceitos

Conceito	O que é
PUBLICATION	Conjunto de tabelas que um servidor disponibiliza para replicação.
SUBSCRIPTION	Assinatura, no outro servidor, que recebe os dados de uma publicação.
wal_level=logical	Nível do WAL necessário para a replicação lógica funcionar.
Replicação seletiva	Só as tabelas dentro da publicação são replicadas.

A replicação lógica copia **apenas os dados** das tabelas escolhidas, não a estrutura (DDL). Por isso as tabelas de destino precisam **existir antes** do assinante, e precisam de **chave primária** para que UPDATE e DELETE sejam replicados.

4. Instalação (passo a passo)

Pré-requisitos: Docker e Docker Compose instalados; portas 5432 e 5433 livres.

1 **Prepare o .env** a partir do exemplo.

```
cp .env.example .env # Windows: Copy-Item .env.example .env
```

2 **Suba o ambiente.** Os 2 servidores sobem e o contêiner setup cria as subscrições automaticamente.

```
docker compose up -d
```

3 **Confira.**

```
docker compose ps
docker compose logs setup # deve mostrar "Replicacao logica configurada"
```

5. Como validar (testes)

A forma mais rápida é o script de demonstração:

```
bash scripts/demo.sh
```

Ele prova os 4 requisitos: mostra os 2 servidores e suas tabelas, as publicações/subscrições, a replicação seletiva (cliente replica, pedido não) e a escrita nos dois lados. Para ver só o estado da replicação: `bash scripts/status.sh`.

6. Passo a passo manual (os comandos por dentro)

Para entender o que acontece, estes são os comandos centrais. As **publicações** são criadas em cada servidor:

```
-- no pg1
CREATE PUBLICATION pub_vendas FOR TABLE clientes, produtos;

-- no pg2
CREATE PUBLICATION pub_avaliacoes FOR TABLE avaliacoes;
```

E as **subscrições** (cada servidor assina a publicação do outro):

```
-- no pg2, assinando o pg1
CREATE SUBSCRIPTION sub_vendas
  CONNECTION 'host=pg1 dbname=bd2 user=replicador password=...'
  PUBLICATION pub_vendas;

-- no pg1, assinando o pg2
CREATE SUBSCRIPTION sub_avaliacoes
  CONNECTION 'host=pg2 dbname=bd2 user=replicador password=...'
  PUBLICATION pub_avaliacoes;
```

Teste A — Replicação seletiva

```
# insere no pg1
docker exec -e PGPASSWORD=admin123 bd2_pg1 psql -U admin -d bd2 \
  -c "INSERT INTO clientes (id,nome,cidade) VALUES (4,'Diego','Xanxere');"
docker exec -e PGPASSWORD=admin123 bd2_pg1 psql -U admin -d bd2 \
  -c "INSERT INTO pedidos (id,cliente_id,total,status) VALUES (3,4,999,'novo');"

# no pg2: o cliente apareceu; pedidos nem existe (esperado falhar)
docker exec -e PGPASSWORD=admin123 bd2_pg2 psql -U admin -d bd2 -c "SELECT count(*) FROM clientes;"
docker exec -e PGPASSWORD=admin123 bd2_pg2 psql -U admin -d bd2 -c "SELECT count(*) FROM pedidos;"
```

Esperado: o cliente novo aparece no pg2; a consulta a `pedidos` no pg2 falha, pois a tabela não foi publicada.

Teste B — Escrita nos dois lados

```
# UPDATE no pg1 replica para o pg2
docker exec -e PGPASSWORD=admin123 bd2_pg1 psql -U admin -d bd2 \
  -c "UPDATE clientes SET cidade='Florianopolis' WHERE id=1;"

# INSERT no pg2 replica para o pg1
docker exec -e PGPASSWORD=admin123 bd2_pg2 psql -U admin -d bd2 \
  -c "INSERT INTO avaliacoes (id,produto_id,nota,comentario) VALUES (3,3,5,'Ótimo');"
docker exec -e PGPASSWORD=admin123 bd2_pg1 psql -U admin -d bd2 -c "SELECT * FROM avaliacoes ORDER BY id;"
```

Esperado: a alteração do pg1 aparece no pg2, e a avaliação inserida no pg2 aparece no pg1.

7. Análise

Vantagens	Desvantagens
Replica só as tabelas escolhidas (seletiva).	Não replica DDL: tabelas criadas à mão no destino.
O destino é um banco gravável.	Exige chave primária para UPDATE/DELETE.
Permite fluxo bidirecional.	Risco de conflito no fluxo bidirecional.
Funciona entre versões diferentes do PostgreSQL.	Mais overhead em volumes muito grandes.

8. Desafios encontrados

Garantir `wal_level=logical` nos dois servidores; criar as tabelas de destino **antes** das subscrições (a replicação lógica não copia a estrutura); criar as subscrições só **depois** que os dois servidores sobem (resolvido com um contêiner `setup` + `depends_on: service_healthy`); e configurar as permissões do usuário de replicação (`REPLICATION` , `SELECT` nas tabelas e liberação no `pg_hba.conf`).

9. Conclusão

O projeto demonstra, de forma reproduzível, a replicação lógica do PostgreSQL com os 4 requisitos atendidos. Resumo: dois servidores independentes trocam tabelas específicas via publicação/subscrição, com replicação seletiva e escrita funcionando nos dois lados.